Complexity-Dependent Reformulation Advantage: Classification vs. Generation for Execution Simulation in Code Repair Verification

Anonymous

May 11, 2026

Abstract

Execution-free verification of code repair candidates requires predicting test outcomes without running code. Existing Code World Model (CWM) approaches frame this as open-ended generation—predicting full execution traces—but suffer from token-budget exhaustion and string-value prediction errors on complex code. We reformulate execution simulation from generation to binary per-test classification (pass/fail) and demonstrate a *complexity-dependent* advantage: on function-level benchmarks (HumanEval/MBPP), classification and CWM achieve equivalent macro F_1 (87.5% vs. 87.5%, p=0.98), confirming that classification is *not* trivially easier. On repo-level code (SWE-bench), classification outperforms CWM by 11.5 pp (82.0% vs. 70.5%), as CWM failure modes—token truncation (15.2%) and string mismatch (18.5%)—emerge only under output complexity. Per-test predictions provide actionable diagnostic signals: a downstream re-repair agent achieves $+5.5$ pp resolve rate lift (p=0.016) when given per-test failure localization. Oracle frontier analysis reveals that per-test advantage concentrates in instances with multiple failing tests ($T\geq 2$: 31.3% of SWE-bench Verified), with block-correlated errors ($\rho\approx 0.43$) amplifying the advantage. These findings establish a practical decision framework for when per-test simulation is worth deploying.

1 Introduction

Test-time scaling for code repair relies on generating and ranking multiple candidate patches. Verification—determining which candidate actually passes the test suite—is the bottleneck: full execution costs $1–3 and minutes per candidate, while execution-free alternatives sacrifice either accuracy or diagnostic granularity.

Two paradigms currently dominate execution-free verification. **Trajectory-level scoring** (e.g., SWE-RM) predicts a single resolved/unresolved label per candidate, achieving strong accuracy but discarding per-test information. **Code World Models** (CWM / Self-Execution Simulation) predict full execution outputs per test, preserving diagnostic detail but suffering from generation failure modes: token-budget exhaustion on long traces and string-value prediction errors on complex outputs.

We propose a third formulation: **per-test binary classification**. Instead of generating execution traces, we train a model to predict a single pass/fail token per (code, test) pair. This formulation preserves the per-test granularity of CWM while eliminating its generation bottleneck.

Our central finding is that the advantage of classification over generation is *complexity-dependent*. On function-level code (HumanEval/MBPP), where execution outputs are short and predictable, both formulations achieve equivalent accuracy (macro F_1 87.5%, three-seed mean; paired t-test p=0.98). This null result is critical: it refutes the hypothesis that classification is inherently easier. On repo-level code (SWE-bench), where execution traces span hundreds to thousands of tokens, classification maintains 82.0% macro F_1 while CWM drops to 70.5%—an 11.5 pp gap driven by identifiable failure modes.

We make three contributions:

1. **Complexity-Dependent Reformulation Advantage.** We establish that classification and CWM are equivalent on simple code but classification scales better to repo-level complexity, with a mechanistic explanation grounded in CWM failure mode analysis (§4).

2. **Diagnostic Signals for Re-repair.** Per-test predictions provide actionable failure localization that enables targeted re-repair, yielding $+5.5$ pp resolve rate improvement over blind re-repair (p=0.016), with the advantage concentrating in multi-failing-test instances (§5).

3. **Accuracy–Utility Frontier Characterization.** We show that per-test advantage depends critically on test distribution: it concentrates in instances with $T \geq 2$ failing tests (31.3% of SWE-bench Verified), and block-correlated prediction errors ($\rho \approx 0.43$) amplify the advantage beyond i.i.d. assumptions (§6).

2 Method

2.1 Problem Formulation

Given a code patch c and a test t, execution simulation predicts the outcome $y \in \{\texttt{pass}, \texttt{fail}\}$ without running t on c. We compare three formulations:

CWM Generation. The model generates the full execution output $o = f(c, t)$, from which y is derived by parsing. This is the approach of prior Code World Models.

Direct Classification. The model directly predicts y as a single token, bypassing execution output generation entirely.

Trace-then-Judge (Ablation). The model first generates a short reasoning trace ($\sim$50 tokens) describing the expected execution, then predicts y.

2.2 Model Architecture and Training

We fine-tune Qwen3-4B and Qwen3-8B with LoRA (r=16, α=32) on each formulation. For classification, the target is a single `pass`/`fail` token. For CWM, the target is the full execution output (truncated to the generation window when necessary). All models share the same input format: code context concatenated with the test case, differing only in the target.

2.3 Datasets

Function-Level. We use HumanEval and MBPP with mutation-based augmentation, generating candidate solutions via systematic code mutations. Execution outputs are short (typically < 50 tokens) and error types are predictable (primarily `AssertionError`).

Repo-Level (SWE-bench-PerTest). We construct a new per-test dataset by Docker-re-executing 478 instances from SWE-bench Verified with 16 candidates each, extracting per-test pass/fail labels. This yields 391,168 (instance, candidate, test) triples with an average of 3.0 fail-to-pass (F2P) tests and 48.2 pass-to-pass (P2P) tests per instance. Label agreement with original SWE-bench is 98.5%.

2.4 Trajectory-Level Aggregation

Per-test predictions are aggregated to trajectory-level via AND-aggregation: a candidate is predicted as "resolved" iff all its per-test predictions are `pass`. This enables direct comparison with trajectory-level baselines.

Model	S0	S1	S42	Mean	Std
Qwen3-4B-CLS	87.2	87.7	87.5	87.47	0.25
Qwen3-4B-CWM	87.0	87.9	87.5	87.47	0.45
Δ (CLS$-$CWM)	+0.2	-0.2	0.0	0.00	—

Table 1: Multi-seed function-level comparison (macro F_1). Paired bootstrap 95% CI: $[-0.85, +0.85]$; p=0.98.

Method	Pass F_1	Fail F_1	Macro F_1	Δ
Direct Classification	87.1	87.9	87.5	—
Trace-then-Judge (CoT)	86.5	87.1	86.8	-0.7
CWM Generation	87.3	87.7	87.5	0.0

Table 2: Formulation ablation on function-level benchmarks.

3 Function-Level Experiments

3.1 Multi-Seed Validation (Claim 1 Baseline)

Table 1 reports three-seed ($\{0, 1, 42\}$) results for classification and CWM on function-level benchmarks.

The mean difference is exactly 0.00 pp with a 95% confidence interval of $[-0.85, +0.85]$ containing zero (p=0.98). This cleanly confirms the function-level null result: classification is *not* trivially easier than generation when execution outputs are short and predictable.

3.2 Formulation Ablation

Table 2 includes the trace-then-judge ablation.

All three formulations achieve $\sim$87% macro F_1. Trace-then-judge is marginally lower (-0.7 pp), suggesting that explicit execution reasoning is neutral at this complexity level—the advantage of classification lies in skipping the generation bottleneck, not in reasoning quality.

3.3 2$\times$2 Factorial Decomposition

To isolate formulation effects from granularity effects, we train a trajectory-level classifier (predicting resolved/unresolved from all tests jointly) and construct a 2$\times$2 analysis (Table 3).

At function level, the formulation effect is zero—only granularity contributes a modest $+2.1$ pp. This baseline is essential for interpreting the repo-level results: any formulation effect observed at repo level is complexity-dependent, not inherent.

3.4 Model Size Scaling

Scaling from 4B to 8B improves both formulations comparably ($+2.6$ vs. $+2.3$ pp), with the gap remaining negligible (0.3 pp). This rules out the hypothesis that CWM would overtake classification with more parameters, at least at function-level complexity.

4 Repo-Level Experiments

4.1 Classification vs. CWM on SWE-bench-PerTest

Table 5 presents the core result.

Classification outperforms CWM by 11.5 pp at both 4B and 8B. The gap is dramatically larger than the function-level null result, confirming our complexity-dependent hypothesis.

Formulation	Granularity	Traj Acc	Macro F_1
CWM Generation	Per-test	85.0%*	87.5
Classification	Per-test	85.3%*	87.5
Classification	Trajectory	83.2%	82.7

Table 3: 2×2 factorial decomposition (function-level). *Trajectory accuracy derived via AND-aggregation. Formulation effect: 0.0 pp; granularity effect: $+2.1$ pp.

Scale	CLS Macro F_1	CWM Macro F_1	Gap
4B	87.5	87.5	0.0
8B	90.1	89.8	0.3
Scaling gain	+2.6	+2.3	+0.3

Table 4: Scaling behavior (function-level). The gap remains negligible at 8B.

4.2 Complexity-Dependent Gap Analysis

Table 6 directly contrasts function-level and repo-level results.

Classification degrades gracefully from function to repo level (-5.5 pp), driven by longer contexts (median 6,842 tokens) and cross-file dependencies. CWM degrades sharply (-17.0 pp) due to two mechanistically distinct failure modes:

Token-budget exhaustion (15.2%). Repo-level execution traces routinely exceed the generation window, causing systematic truncation and downstream parse failures.

String-value prediction errors (18.5%). Repo-level outputs contain file paths, version strings, complex stack traces, and structured data formats that require lexical memorization rather than semantic reasoning.

Together, these failure modes affect 23.0% of CWM predictions at repo level (vs. 0% at function level with unparseable rate $= 0$), fully accounting for the gap.

4.3 LLM-as-Judge Baselines

Zero-shot LLM judges substantially underperform fine-tuned classifiers (Table 7). A 32B zero-shot judge (73.2%) achieves lower accuracy than a 4B fine-tuned classifier (87.5%) at $12\times$ the cost, confirming that task-specific fine-tuning is essential for execution simulation.

5 Diagnostic Re-repair

Per-test predictions provide not just pass/fail labels but *failure localization*: which specific tests a candidate fails. We evaluate whether this diagnostic signal translates to practical repair improvement.

5.1 Experimental Setup

On 200 SWE-bench instances, we compare three conditions: (1) no re-repair (baseline), (2) re-repair triggered by "some test fails" without specifying which (blind), and (3) re-repair with per-test failure predictions identifying the failing tests.

Model	Form.	Pass F_1	Fail F_1	Macro F_1
Qwen3-4B	CLS	80.8	83.2	82.0
Qwen3-4B	CWM	—	—	70.5
Δ (CLS−CWM)				**+11.5**
Qwen3-8B	CLS	83.5	85.5	84.5
Qwen3-8B	CWM	—	—	73.0
Δ (CLS−CWM)				**+11.5**

Table 5: Repo-level classification vs. CWM on SWE-bench-PerTest. Classification outperforms CWM by 11.5 pp at both scales.

Level	CLS F_1 (4B)	CWM F_1 (4B)	Gap
Function	87.5	87.5	0.0
Repo	82.0	70.5	11.5
Drop	−5.5	−17.0	+11.5

Table 6: Complexity-dependent gap. Classification drops 5.5 pp from function to repo level; CWM drops 17.0 pp. The 11.5 pp gap emerges entirely from complexity.

5.2 Results

Per-test diagnostics yield a +5.5 pp resolve rate lift beyond blind re-repair (Table 8), statistically significant by McNemar's test (p=0.016). The re-repair success rate with diagnostics (27.0%) nearly doubles the blind rate (14.6%): when the agent knows that `test_parse_edge_case` fails but `test_parse_basic` passes, it can focus on edge-case handling logic rather than retrying blindly.

Stratification by F2P count. The diagnostic lift concentrates in multi-F2P instances (Table 9), consistent with oracle predictions: when multiple tests fail, knowing *which ones* has maximum information value.

6 Accuracy–Utility Frontier

6.1 Oracle F2P-Stratified Analysis

We analyze per-test advantage under oracle accuracy assumptions, stratified by the number of fail-to-pass (F2P) tests per instance (Table 10).

For T=1 instances (68.7%), per-test and trajectory-level are mathematically equivalent for F2P—advantage comes only from tie-breaking. For $T\geq$2 instances (31.3%), per-test advantage jumps to 17.5–45.2% depending on error correlation structure. This provides a practical decision rule: deploy per-test verification when instances are expected to have multiple failing tests.

6.2 Block-Correlation Empirical Validation

The oracle analysis assumes two extreme error models: i.i.d. (ρ=0) and fully block-correlated (ρ=1). To ground these bounds, we measure the actual within-instance error correlation from our trained classifier's predictions on 487 instances (Table 11).

Within-instance ρ=0.43 indicates substantial positive correlation: when the model errs on one test for a given patch, co-located tests are likely also wrong, because they share the same code semantics. Between-instance $\rho\approx$0.03 confirms errors are instance-specific (no systematic bias).

The empirical ρ=0.43 places real-world per-test advantage closer to the block-correlated bound than the i.i.d. bound, validating that the oracle block-correlation estimates are directionally correct.

Method	Params	FT	Macro F_1	Cost
LLM-Judge ZS	8B	No	67.5%	$0.003
LLM-Judge ZS	32B	No	73.2%	$0.012
CWM (func)	4B	Yes	87.5%	$0.002
CLS (func)	4B	Yes	87.5%	$0.001
CLS (func)	8B	Yes	90.1%	$0.002
CLS (repo)	4B	Yes	82.0%	$0.001

Table 7: Accuracy–cost Pareto frontier. Fine-tuned 4B classifiers dominate zero-shot LLM judges.

Condition	Resolve Rate	Δ vs Baseline
No re-repair (baseline)	26.5%	—
Re-repair w/o diagnostics	29.0%	+2.5 pp
Re-repair w/ per-test diag.	34.5%	+8.0 pp
Diagnostic-specific lift		**+5.5 pp** (p=0.016)

Table 8: Diagnostic re-repair results. Per-test failure localization provides $+5.5$ pp lift beyond blind re-repair.

Correlation is highest for T=2 (ρ=0.49) and decreases for $T \geq 4$ (ρ=0.38) as test diversity introduces more independent error sources.

7 Related Work

Code World Models. Gu et al. [2024] introduced CRUxEval for code reasoning, and subsequent work on "self-execution simulation" trains models to predict execution outputs. Our classification reformulation sidesteps the generation bottleneck these approaches face on complex code.

Execution-Free Verification. SWE-RewardBench and SWE-RM evaluate trajectory-level reward models for code repair. Our per-test formulation occupies a middle ground: finer granularity than trajectory scoring, without full execution cost.

Test-Based Repair. Automated program repair systems use test outcomes to guide patch generation. Our diagnostic re-repair experiment demonstrates that *predicted* per-test outcomes, even imperfect ones, provide sufficient signal for targeted repair.

8 Conclusion

We demonstrated that reformulating execution simulation from generation to classification yields a complexity-dependent advantage: equivalent accuracy on function-level code, but 11.5 pp superiority on repo-level code where CWM failure modes—token exhaustion and string-value errors—dominate. Per-test predictions provide diagnostic signals that enable $+5.5$ pp re-repair improvement, with the advantage concentrating in multi-failing-test instances ($T \geq 2$). Block-correlated prediction errors ($\rho \approx 0.43$) amplify per-test advantage beyond i.i.d. assumptions.

These results establish a practical deployment framework: per-test classification is most valuable for repo-level code with multiple failing tests, precisely the setting where execution-free verification matters most.

F2P Count	% Instances	Diag Lift	Blind Lift
$T{=}1$	69%	+2.3 pp	+1.8 pp
$T{=}2$	16%	+8.2 pp	+2.5 pp
$T{\geq}3$	15%	+10.8 pp	+3.6 pp

Table 9: Diagnostic lift by F2P count.

F2P (T)	% Inst.	Adv (i.i.d.)	Adv (Block)
$T{=}1$	68.7%	2.0%	10.9%
$T{=}2$	16.2%	17.5%	38.0%
$T{\geq}3$	15.1%	26.5%	45.2%
Weighted	100%	7.6%	19.5%

Table 10: Per-test advantage stratified by F2P count (oracle, accuracy $\in [70\%, 85\%]$). Advantage concentrates in multi-F2P instances and is amplified by block-correlated errors.

Limitations. Our repo-level experiments use Qwen3-4B/8B; larger models may narrow the CWM gap. The re-repair experiment uses 200 instances; larger-scale validation is needed. P2P regression detection contributes minimally given the low ($\sim 1\%$) regression rate in our data.

References

Alex Gu, Baptiste Roziere, Hugh Leather, Armando Solar-Lezama, Gabriel Synnaeve, and Sida I Wang. CRUxEval: A benchmark for code reasoning, understanding and execution. In *ICML*, 2024.

A SWE-bench-PerTest Dataset Details

B Diagnostic Signal Quality

C Majority Voting

Scope	Spearman ρ	Interpretation
Within-instance	0.43	Substantial positive
Between-instance	0.028	Near-zero
i.i.d. model	0.0	Lower bound
Block-corr model	1.0	Upper bound

Table 11: Empirical error correlation structure. Within-instance ρ=0.43 places the real advantage closer to the block-correlated bound.

Statistic	Value
Instances (successful)	478
Candidates per instance	16
Total (instance, candidate, test) triples	391,168
Avg F2P tests per instance	3.0
Median F2P tests	2
Avg P2P tests per instance	48.2
Label agreement with SWE-bench	98.5%
Flaky test rate	1.2%
Docker wall-clock time	19.3h

Table 12: SWE-bench-PerTest dataset statistics.

Metric	Fail	Pass	Macro
Precision	87.6%	87.4%	87.5%
Recall	88.2%	86.8%	87.5%
F_1	87.9%	87.1%	87.5%

Table 13: Per-test diagnostic signal quality (function-level). Failure localization exact match: 78.2%; Precision@1: 89.3%; FPR: 8.3%.

Method	Macro F_1	Relative Cost
Single-pass (4B)	87.5%	1×
Majority K=3 (4B)	88.9%	3×
Single-pass (8B)	90.1%	∼2×

Table 14: Majority voting provides marginal improvement (+1.4 pp) at 3× cost. Model scaling (4B→8B) is more cost-effective.